

UE19CS251

DESIGN AND ANALYSIS OF ALGORITHMS

UNIT 5: Limitations of Algorithmic Power and
Coping with the Limitations

Transitive Closure (Warshall's
Algorithm)

PES University

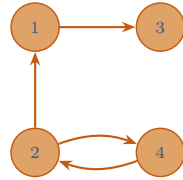
Outline

Concepts covered

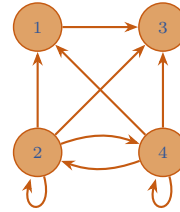
- Transitive Closure (Warshall's Algorithm)
 - Motivation
 - Algorithm
 - Example

1 Transitive Closure

- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph (directed graph)
- Example of transitive closure:



$$\begin{array}{c}
 \begin{array}{cccc}
 & 1 & 2 & 3 & 4 \\
 \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \end{array} & \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}
 \end{array}
 \end{array}$$



$$\begin{array}{c}
 \begin{array}{cccc}
 & 1 & 2 & 3 & 4 \\
 \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \end{array} & \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}
 \end{array}
 \end{array}$$

2 Warshall's Algorithm

- Constructs transitive closure T as the last matrix in the sequence of $n \times n$ matrices $R^{(0)}, \dots, R^{(k)}, \dots, R^{(n)}$ where $R^{(k)}[i, j] = 1$ iff there is nontrivial path from i to j with only first k vertices allowed as intermediate vertices
 - $R^{(0)} = A$ (adjacency matrix), $R^{(n)} = T$ (transitive closure)
- On the k^{th} iteration, the algorithm computes $R^{(k)}$

$$R^{(k)}[i, j] = \begin{cases} 1 & \text{if path from } i \text{ to } k \text{ and } k \text{ to } j, \text{ i.e., } R^{(k-1)}[i, k] = R^{(k-1)}[k, j] = 1 \\ R^{(k-1)}[i, j] & \text{otherwise} \end{cases}$$

$$R^{(k)}[i, j] = R^{(k-1)}[i, j] \text{ or } (R^{(k-1)}[i, k] \text{ and } R^{(k-1)}[k, j])$$

3 Algorithm

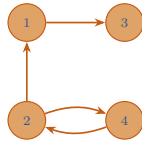
Transitive Closure (Warshall's Algorithm)

```

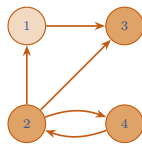
1: procedure WARSHALL( $A[1 \dots n, 1 \dots n]$ )
2:    $\triangleright$  Input: The adjacency matrix  $A$  of a digraph with  $n$  vertices
3:    $\triangleright$  Output: The transitive closure of the digraph
4:    $R^{(0)} \leftarrow A$ 
5:   for  $k \leftarrow 1$  to  $n$  do
6:     for  $i \leftarrow 1$  to  $n$  do
7:       for  $j \leftarrow 1$  to  $n$  do
8:          $R^{(k)}[i, j] \leftarrow R^{(k-1)}[i, j] \text{ or } (R^{(k-1)}[i, k] \text{ and } R^{(k-1)}[k, j]);$ 
9:   return  $R^{(n)}$ 

```

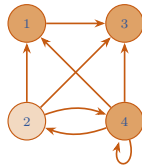
4 Warshall's Algorithm



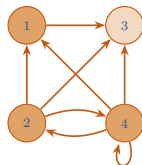
	1	2	3	4
1	0	0	1	0
2	1	0	0	1
3	0	0	0	0
4	0	1	0	0



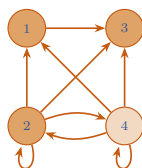
	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	0	1	0	0



	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	1	1	1	1



	1	2	3	4
1	0	0	1	0
2	1	0	1	1
3	0	0	0	0
4	1	1	1	1



	1	2	3	4
1	0	0	1	0
2	1	1	1	1
3	0	0	0	0
4	1	1	1	1

5 Think About It

- Is Warshall's algorithm efficient for sparse graphs? Why / why not?
- Can Warshall's algorithm be used to determine if a graph is a DAG (Directed Acyclic Graph)? If so, how?